



中国科学院大学
University of Chinese Academy of Sciences

研究生学位论文开题报告

报告题目	面向低延迟大语言模型推理的算子融合决策与 跨 Kernel 协同优化研究		
学生姓名	郝朝阳	学号	待填写
指导教师	待填写	职称	待填写
学位类别	工学硕士		
学科专业	计算机科学与技术		
研究方向	AI 编译器与高性能计算		
培养单位	处理器芯片全国重点实验室		
填表日期	2026 年 07 月		

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 选题背景	1
1.2 选题意义	4
二 国内外本学科领域的发展现状与趋势	5
2.1 LLM 推理服务与资源管理	5
2.2 深度学习编译与算子融合	6
2.3 大范围融合与设备侧整体执行	7
2.4 图重放、多流并行与跨 Kernel 重叠	8
2.5 现有工作小结	9
三 课题主要研究内容与预期目标	10
3.1 总体目标	10
3.2 拟解决的关键问题	11
3.3 主要研究内容	11
3.4 预期创新点	12
四 拟采用的研究方法、技术路线、实验方案及可行性分析	13
4.1 总体研究方法	13
4.2 边界表示与分析方法	13
4.3 选择性算子融合方法	13
4.4 依赖感知跨 Kernel 边界的 DACOS 方法	14
4.5 协同决策与运行时	15
4.6 实验方案	15
4.7 可行性分析	17
五 已有科研基础与所需科研条件	17
5.1 已有科研基础	17
5.2 所需科研条件	18

六 研究工作计划与进度安排	18
---------------------	----

一 选题的背景及意义

1.1 选题背景

大语言模型（Large Language Model, LLM）通常采用自回归方式生成文本。一次推理请求首先处理输入序列并建立 Key-Value Cache（KV Cache），随后逐 token 生成输出。现有综述通常将 LLM 推理系统的优化划分为算子与内核实现、模型执行、批处理与调度、KV Cache 管理以及分布式部署等层次；这些层次共同决定系统的吞吐、时延和资源效率Li 等 (2024a); Pan 等 (2025); Li 等 (2024b)。

与训练过程相比，LLM 在线推理具有两个直接影响系统设计的特征。一方面，模型参数规模和 KV Cache 使内存容量、带宽及跨设备通信成为基础约束；另一方面，自回归生成在 token 维度上具有串行依赖，一个 token 的完成时间会直接推迟后续 token 的生成。因而，推理优化不能只追求离线吞吐，还需要在给定服务等级目标（Service Level Objective, SLO）下控制首 token 和后续 token 的响应时间。算子实现、模型执行和服务调度分别作用于不同时间尺度：算子实现决定单次计算的效率，模型执行决定一层或一次迭代中的关键路径，服务调度则决定多个请求如何共享设备。本课题关注其中的模型执行层，但其性能评价仍需放回完整请求的时延指标中。

从请求执行过程看，生成式 LLM 推理可分为预填充（prefill）和解码（decode）两个阶段。Prefill 对输入 token 进行并行计算，并产生首个输出 token；decode 则依据已经保存的 KV Cache，每次为每个请求生成一个新 token。Sarathi-Serve 指出，prefill 具有较高的单次计算量，通常能够更充分地利用 GPU，而 decode 单步计算量较小，需要借助批处理提高设备利用率Agrawal 等 (2024)。相应地，在线服务通常以首 token 时延（Time to First Token, TTFT）、单输出 token 时延（Time per Output Token, TPOT）、端到端时延以及满足时延约束的有效吞吐作为主要性能指标Zhong 等 (2024); Patel 等 (2024)。

实际在线负载并不能简单归结为“短 prompt、短 decode”。对话、检索增强生成、代码补全和智能体任务会形成差异显著的输入长度、输出长度和到达模式；多轮调用还可能携带增长的会话上下文，或者通过 prefix/KV Cache 复用避免重复计算。智能体程序进一步在模型调用之间插入检索、工具执行和控制逻辑，使一次用户任务表现为多次相互依赖的模型调用Luo 等 (2026); Sui 等 (2026); He 等

(2026); [Bian 等 \(2025\)](#)。因此，本课题不把某一应用类型直接等同于短序列，而是将 batch、prompt length、KV length 和生成长度作为独立变量。其关注的低时延区域主要包括小 batch decode、短增量 prefill，以及工具调用前后需要快速返回的模型阶段；这些阶段的共同特点是单次设备工作量较小，固定提交成本和细粒度执行开销更难被摊薄。

在模型执行层，Transformer 会被分解为矩阵乘、Attention、归一化、位置编码、激活和残差计算等算子，并进一步映射为一系列 GPU Kernel。算子之间通过中间张量形成生产者-消费者关系。逐算子执行具有清晰的模块边界，便于复用高性能算子库，但中间结果通常需要写入全局内存并由后继 Kernel 重新读取，同时还会产生 Kernel 启动和同步开销。对低 batch 推理，单个算子的计算规模减小，数据移动和启动开销在总执行时间中的比例会更加明显；FlashFormer 和相关工作正是将低 batch 下的内存带宽与 Kernel 启动开销视为重要优化对象[Nrusimha 等 \(2025\)](#); [Vellaisamy 等 \(2025\)](#)。

从时间线看，一次模型迭代的延迟不仅由各 Kernel 的执行时间相加得到，还包括主机侧算子包装与参数准备、Kernel 提交间隔、设备侧依赖等待以及同步造成的流水空隙。不同开销具有不同消除方式：Python 或框架调度开销可以通过图捕获、降低框架层级或设备侧调度减少；中间张量物化主要依赖融合和数据流重构；真实生产者-消费者依赖则不能通过把任务放入不同 stream 自动消失。近期工作开始将框架、运行时和硬件执行的附加成本单独分解，说明在快速 Kernel 占比较高时，端到端性能可能受计算之外的固定成本限制[Vellaisamy 等 \(2026\)](#)。这要求优化方法先辨别边界的成因，再选择与之匹配的变换。

算子融合是减少上述开销的主要技术之一。深度学习编译器通过图变换和循环变换将多个算子合并到同一 Kernel 中，使中间数据保留在寄存器、共享内存或其他片上存储层级，从而减少全局内存访问和 Kernel 启动。TVM 将算子融合列为图级优化的重要组成部分；DNNFusion 从算子属性和图重写出发扩展融合范围；Welder 进一步使用 tile-level 数据流量模型，在算子内与算子间数据复用之间进行联合权衡[Chen 等 \(2018\)](#); [Niu 等 \(2021\)](#); [Shi 等 \(2023\)](#)。在 Transformer 计算中，FlashAttention 通过分块、在线 Softmax 和算子融合避免显式物化完整 Attention 矩阵，说明融合还可以与算法和数据流重构共同设计[Dao 等 \(2022\)](#); [Dao \(2023\)](#); [Shah 等 \(2024\)](#)。

然而，“能够融合”并不等价于“融合后一定更快”。扩大融合区域可以减少启动和中间访存，却也会延长变量生命周期，增加寄存器和共享内存占用，并可能降低 occupancy、限制线程块调度或破坏成熟库 Kernel 的高效实现。Reduction、GEMM 和 elementwise 算子的迭代空间不同，融合还需要处理跨线程同步、数值归约顺序和数据布局转换。动态 batch、序列长度、KV length 以及 mask 类型会进一步影响代码特化：形状可符号化时可以由 guard 和 shape bucket 覆盖，而由输入值决定控制流或输出规模时，编译器可能需要 graph break、重新编译或回退。因而，面向 LLM 的融合研究不应只枚举模式，而需要回答融合区域如何形成、何时停止扩张，以及何时主动保留一个可优化的 Kernel 边界。

除消除 Kernel 边界外，现代 GPU 也提供了在保留 Kernel 结构时调整执行次序的机制。CUDA Graph 通过捕获和重放执行图降低重复提交开销；Programmatic Dependent Launch (PDL) 允许存在依赖关系的后继 Kernel 在前驱完全结束前启动，并在读取依赖数据前进行同步NVIDIA (2026a,c)。这使得后继 Kernel 中与前驱输出无关的准备工作有机会与前驱尾部执行重叠，但其正确性和收益取决于依赖位置、可提前工作量以及两个 Kernel 的资源竞争。

保留 Kernel 边界并不只有“融合失败”这一种原因。边界可能来自外部高性能库、不同迭代空间、动态控制流、编译与维护成本，也可能是为了避免过大的资源占用而由优化器主动保留。对这些边界，完全独立的任务可以采用多 stream 并行，稳定的重复序列可以使用 CUDA Graph 降低提交开销；对于具有真实数据依赖、但消费者包含独立前缀的相邻 Kernel，则存在设备侧提前启动和依赖后移的空间。该类优化不会消除中间张量，也不能替代融合带来的数据局部性，但能够在保持已有 Kernel 实现的同时压缩关键路径上的等待，因此构成另一类执行图变换。

基于上述研究基础，本课题面向低时延 LLM 推理中的 GPU 执行图，研究算子融合与依赖感知的跨 Kernel 执行优化。研究内容包括：识别具有数据复用价值的融合区域，分析融合粒度对数据移动和硬件资源的影响；对执行图中保留的生产者-消费者边，识别后继 Kernel 中可提前执行的计算并构造跨 Kernel 重叠；结合运行时形状与资源状态选择执行方案。课题重点位于单次模型执行内部，与批处理、KV Cache 管理和集群调度等服务级优化形成互补。

1.2 选题意义

首先，本课题有助于深化对 LLM 推理执行开销的认识。算子级性能分析主要刻画单个 Kernel 的计算量、访存量和并行度，而模型时延还受到算子间数据移动、启动、同步和依赖等待的共同影响。将算子内部执行与算子间边界统一纳入分析，可以更准确地解释局部 Kernel 性能与端到端时延之间的差异，并为融合粒度和执行次序的选择提供依据。

这一分析视角的意义在于把“边界”从时间线中的空隙转化为可描述、可比较的优化对象。同一条生产者-消费者边可以同时具有中间数据量、启动成本、依赖位置、资源占用和动态特化风险等属性；不同属性分别对应融合、图重放、多流或依赖感知重叠。建立从编译图语义到设备时间线的映射，有助于避免仅根据 Kernel 数量或单个算子的加速比判断端到端收益，也为解释负优化和确定方法适用范围提供统一基础。

其次，本课题有助于完善面向 GPU 执行图的优化方法。算子融合通过扩大数据局部性减少中间结果物化，跨 Kernel 重叠则在保留模块边界的条件下利用可提前执行的计算。联合考虑两类变换，可以在数据复用、并行度、片上资源占用和动态适应性之间进行权衡，为低时延推理提供比单一融合规则或固定执行顺序更完整的优化空间。

其中，融合部分的研究价值不在于重复实现已有的 `elementwise` 模板，而在于面向 LLM 常见结构建立具有资源约束的融合分区方法：既考察残差、归一化、线性层前后处理、Attention wrapper 等局部数据复用，也把成熟库性能、动态形状和代码版本成本纳入决策。跨 Kernel 部分则研究依赖消费者内部的阶段划分和同步位置，使“边界被保留”之后仍存在可分析的优化空间。二者共同形成从图分区到边界调度的连续决策过程。

最后，本课题具有实际工程价值。LLM 推理框架通常同时依赖编译器生成 Kernel、高性能算子库和 GPU 运行时机制。围绕现有软件栈开展融合和跨 Kernel 优化，可以在不改变模型语义和服务接口的前提下降低单次模型执行时延，并为不同 batch、序列长度和硬件资源条件提供可回退的执行方案。该方向可与连续批处理、KV Cache 管理和 prefill/decode 解耦等服务级技术组合，进一步改善在线推理的性能与资源效率。

此外，研究结论可以为编译器和推理框架提供更具体的工程反馈：哪些边界

应通过代码生成消除，哪些边界更适合调用专用库，哪些稳定序列适合图重放，以及哪些依赖边可以利用新硬件机制提前执行。将这些选择记录为可验证、可回退的执行计划，比对整个模型采用单一优化策略更符合现有 LLM 软件栈由框架、编译器、算子库和运行时共同组成的现实。

二 国内外本学科领域的发展现状与趋势

2.1 LLM 推理服务与资源管理

LLM 推理服务首先面临自回归生成带来的动态批处理问题。不同请求具有不同的输入和输出长度，并会在不同迭代进入或离开执行批次。Orca 提出的 iteration-level scheduling 允许系统在迭代边界重新组织 batch，提高了请求并发执行的灵活性Yu 等 (2022)。vLLM 通过 PagedAttention 将 KV Cache 按块管理，减少内存碎片和冗余预留，从而提高可用于批处理的显存容量Kwon 等 (2023)。这两类工作分别从调度粒度和状态存储方式奠定了现代 LLM serving engine 的基础。

后续研究进一步围绕 prefill/decode 的执行差异优化资源分配。Sarathi-Serve 采用 chunked prefill 构造更均匀的批次，以缓解长 prefill 对正在执行的 decode 请求造成的停顿Agrawal 等 (2024)。DistServe 和 Splitwise 将 prefill 与 decode 放置在不同资源上，分别针对 TTFT 和 TPOT 配置并行策略Zhong 等 (2024); Patel 等 (2024)。Mooncake 则以 KV Cache 为中心组织分离式资源，并将 CPU、内存、SSD 和网络共同纳入缓存管理Qin 等 (2025)。总体上，LLM serving 已从单实例内的连续批处理发展到面向阶段差异、缓存状态和 SLO 的资源协同。

面向多轮与智能体应用的研究继续扩大了调度对象。Agentix 将智能体工作流程表示为包含模型调用和外部函数的程序，PASTE 研究工具执行与后续生成之间的并行，AMPD 和 Tokencake 则分别关注多轮推理与多智能体场景下的 KV Cache 复用和传递Luo 等 (2026); Sui 等 (2026); He 等 (2026); Bian 等 (2025)。这些系统表明，请求调度正在从单个 prompt-response 对扩展为带状态、依赖和外部事件的执行过程。不过，这类系统优化主要决定请求何时运行、状态存放在哪里以及不同阶段使用哪些资源；当调度器已经选定一个 batch 后，其内部 Transformer 层仍需要通过一系列 GPU Kernel 完成。因此，服务级调度与本课题关注的模型执行优化是上下层关系，不能用前者代替对设备关键路径的分析。

2.2 深度学习编译与算子融合

深度学习编译器通常在计算图和算子实现两个层次开展优化。TVM 使用图级变换处理算子融合和数据布局，并通过自动调优生成面向不同硬件的算子实现Chen 等 (2018)；Triton 提供面向分块张量计算的编程抽象，使开发者能够显式组织数据分块、并行映射和存储层级Tillet 等 (2019)；TorchInductor 则从 PyTorch 图生成融合 Kernel，并结合 Triton 或外部算子库执行PyTorch Contributors (2026b)。这些系统使 elementwise chain、广播、类型转换和部分 reduction 周边操作能够在编译阶段合并。

从变换方式看，现有融合大致包括基于计算图模式的纵向融合、共享输入或相似调度的横向融合，以及将轻量操作吸收到矩阵乘或卷积 epilogue/prologue 的库内融合。图模式方法实现稳定、便于维护，但覆盖范围受预定义规则限制；循环或 tile 级方法能够发现更细的数据复用，却需要解决迭代空间对齐、同步与存储规划；库内融合通常保留主算子的高效实现，但可接入的前后处理种类和数据布局受到接口约束。TensorRT 的层融合目录和 CUTLASS GEMM 接口体现了工程系统中“图重写、专用库和代码生成并存”的实现方式NVIDIA (2026d,b)。

围绕融合范围与数据移动，已有工作提出了更细致的图表示和代价模型。DNNFusion 根据算子的数学属性分类，通过图重写、融合计划生成和轻量 profiling 扩展可融合组合Niu 等 (2021)。Welder 以 tile-graph 表示跨算子数据流，并以不同存储层级上的数据流量为依据，联合选择 tile 和融合方案Shi 等 (2023)。这些工作表明，融合决策不仅取决于图结构是否合法，还取决于张量形状、数据复用、片上资源以及生成代码的执行效率。

融合合法性和融合收益是两个不同问题。合法性要求变换保持别名关系、读写顺序、随机数状态和数值语义；收益判断则需要估计被消除的中间访存与启动成本，并扣除重复计算、同步、寄存器压力和 occupancy 下降。对静态计算图，这些信息可以在编译期分析并通过离线调优校准；对 LLM serving，连续批处理导致 batch 变化，KV Cache 长度随生成推进，Attention backend 还可能按形状切换。编译器因此需要符号形状、guard、bucket 或多版本代码，而 value-dependent mask、动态输出规模和 Python 控制流可能造成 graph break。现有系统已经分别处理图捕获、融合和动态形状，但如何把动态适应成本纳入融合区域选择，仍是面向在线推理的重要问题。

Attention 是 LLM 中算子融合与算法重构结合的代表。FlashAttention 通过 IO-aware tiling 和在线 Softmax，在不物化完整 Attention 矩阵的情况下完成分块计算；FlashAttention-2/3 进一步改进线程块与 warp 之间的工作划分、异步数据移动和低精度执行Dao 等 (2022); Dao (2023); Shah 等 (2024)。FlashInfer 则针对 serving 中的不同 Attention 形状和 KV Cache 组织提供可定制 KernelYe 等 (2025)。这一发展路径说明，面向 LLM 的融合已从简单逐元素合并扩展到 reduction、矩阵乘和内存调度的联合设计。

不过,FlashAttention 一类成功方案依赖对特定数学结构的重新推导和专门实现，其经验不能直接转化为“尽可能扩大所有算子融合范围”。在 Transformer 的非 Attention 区域,矩阵乘通常由高度调优的库 Kernel 完成,归一化和 elementwise 操作则具有不同的并行与归约结构；将它们合并可能减少中间读写，也可能损失主算子的调度效率。因而，当前研究趋势一方面继续发展专用 fused kernel，另一方面也在探索能够比较多个融合粒度的编译表示和成本模型。

2.3 大范围融合与设备侧整体执行

为进一步减少 Kernel 启动和算子间数据传输，部分研究开始扩大设备侧执行区域。ClusterFusion 利用 Hopper GPU 的 thread-block cluster 和分布式共享内存,将 QKV projection、Attention 与 output projection 组织为 cluster-level fused kernelLuo 等 (2025)。FlashFormer 面向低 batch 推理构造 whole-model kernel，以减少模型执行过程中的启动和内存访问Nrusimha 等 (2025)。Mirage Persistent Kernel (MPK) 将张量程序下降为 SM-level task graph，并在 persistent kernel 内进行设备侧调度和跨算子流水Cheng 等 (2025)；Event Tensor 则使用显式事件描述动态依赖，以支持包含动态形状和数据依赖的 megakernel 编译Jin 等 (2026)。

上述研究体现出从局部融合向 cluster-level、block-level 乃至 whole-model execution 扩展的趋势。随着融合区域增大，编译器需要同时处理算子间依赖、线程块协作、寄存器与共享内存分配以及运行时动态性，因此融合范围和执行计划通常需要结合具体模型、输入形状和硬件资源确定。

整体式执行的核心变化不是简单减少 Kernel 数量，而是把原本由 CUDA runtime 在 Kernel 边界完成的全局调度，部分转移到 persistent kernel 内部。这样可以复用常驻线程块、以片上队列表达细粒度任务，并在设备侧直接衔接多个算

子；相应代价是编译器和运行时需要自行处理 SM 间负载均衡、全局同步、资源常驻和异常路径。模型规模、专家路由、动态序列和条件分支都会改变任务图，仅依赖静态 megakernel 难以覆盖所有在线输入。Event Tensor 和 Ada-MK 分别从动态依赖表示和自动搜索角度处理这一问题Jin 等 (2026); Dong 等 (2026)，反映出大范围融合正在从手工整体 Kernel 走向编译器与运行时协同。

从适用范围看，whole-model 或大范围 persistent kernel 在小 batch、稳定模型结构以及固定硬件目标上具有明显吸引力，但它对代码生成能力、资源规划和模型适配提出较高要求。生产推理栈仍会长期同时包含编译生成 Kernel、cuBLAS/CUTLASS 等算子库以及不可捕获的动态路径。因此，大范围融合可以作为重要性能上界和候选方案，却不能消除研究局部融合决策与保留边界执行方式的必要性。

2.4 图重放、多流并行与跨 Kernel 重叠

CUDA Graph 将一组 GPU 操作及其依赖捕获为可重复提交的图，以降低重复执行路径中的 CPU 提交开销NVIDIA (2026a); PyTorch Contributors (2026a); vLLM Contributors (2026)。对于具有稳定形状和控制流的 decode 路径，图重放已经成为常用的运行时优化。多流执行则利用不同 CUDA stream 调度没有数据依赖或仅具有显式事件依赖的任务，以提高设备并发度。这些机制主要改变 Kernel 的提交和调度方式，并不直接改变单个 Kernel 内部的数据流。

CUDA Graph 的收益来源与算子融合并不相同。图重放能够压缩 host-side launch path，但图中的每个 Kernel 仍保持原有全局内存交互和设备侧依赖；融合能够消除部分边界，却可能增加代码特化和资源压力。实际框架通常对稳定 shape bucket 捕获多个图，并在输入形状、控制流或内存地址不满足捕获约束时回退。因而，评价一个新的设备执行优化时，需要在启用 CUDA Graph 的共同基线上继续观察 GPU timeline，而不能把 eager 模式下减少 Python 开销的收益计入 Kernel 变换。

多流并行同样受依赖关系限制。两个无依赖分支可以由不同 stream 并发提交，但生产者输出被消费者读取时，事件同步仍要求消费者的相关工作等待数据就绪。如果消费者在读取该输出之前还包含地址计算、描述符设置、独立输入加载或其他准备操作，传统事件通常以整个 Kernel 为同步粒度，无法表达 Kernel

内部“先执行独立前缀、再等待依赖数据”的阶段关系。这一粒度差异构成 PDL 和依赖感知跨 Kernel 优化的切入点。

对于存在生产者-消费者关系的相邻 Kernel，传统同流执行需要等待前驱完成。NVIDIA PDL 允许后继 Kernel 在前驱完全结束前启动；后继可先执行不读取前驱输出的操作，并在依赖数据首次使用前调用 `grid-level synchronization` NVIDIA (2026c)。官方文档同时指出，提前启动和并发执行是机会性的，其效果取决于可重叠工作以及两个 Kernel 的资源占用。

DACOS 在 PDL 机制上进一步引入跨 Kernel 依赖分析、算子相关的 overlap stage 和配置选择 Hao 等 (2026)。该方法区分后继 Kernel 的依赖输入与独立输入，将地址计算、描述符设置和独立数据预取等操作移动到同步点之前，并联合选择触发阶段、预取量和存储层级。与 CUDA Graph 主要减少 host-side launch overhead 不同，DACOS 关注保留 Kernel 边界时的 device-side dependent execution。

这一方向目前仍处于相对早期阶段。PDL 提供的是机会性执行语义，硬件并不保证两个 Kernel 一定形成有效并发；前驱占满 SM、消费者独立前缀过短或提前加载造成缓存与共享内存竞争时，重叠可能没有收益。与融合相比，跨 Kernel 重叠保留了中间数据物化，却能够继续复用已有高性能 Kernel，并避免大融合区域的寄存器生命周期扩张。因而，它适合以边界为单位进行合法性证明、资源分析和运行时准入，而不是对所有相邻 Kernel 统一开启。

2.5 现有工作小结

现有 LLM 推理优化已覆盖服务调度、KV Cache 管理、编译器融合、专用 Kernel、megakernel 和图重放等多个层次。服务级方法主要改变请求组织和资源分配，算子融合与整体式执行主要减少数据移动和 Kernel 边界，图与多流机制主要降低提交成本或利用任务间并行。不同层次可以组合使用，但其优化对象和适用条件并不相同。

算子融合方面，研究已经证明减少中间数据传输和扩大片上复用能够显著改善多类 DNN 与 Transformer 计算；当前不足不在于缺少融合概念，而在于融合决策仍容易被简化为固定模式或“合法即融合”。面向 LLM 在线推理，需要进一步考虑成熟库调用、动态 shape、mask 语义、编译版本数以及融合后的资源占用，并在局部融合、较大融合和保留边界之间形成可解释的选择。该问题决定本

课题融合工作的研究空间，也要求实验与 TorchInductor、专用 fused library 和较大范围执行方案比较，而不能只以 eager execution 为基线。

保留边界方面，CUDA Graph、多流和 PDL 分别处理提交重放、独立任务并行和依赖 Kernel 的提前启动，它们作用于不同开销。现有框架对前两类机制已有较多应用，但对消费者 Kernel 内部依赖位置和独立工作量的分析仍不充分。DACOS 为这一问题提供了前期方法基础，进一步工作需要说明哪些边界满足重叠条件、融合改变图分区后机会如何迁移，以及资源竞争和动态输入下何时回退。

从 GPU 执行图角度看，仍需进一步研究两个相互关联的问题：一是如何根据算子依赖、数据复用和硬件资源确定合适的融合区域，而非仅依据固定模式扩大融合范围；二是对执行计划中保留的依赖边，如何识别后继 Kernel 的可提前阶段并判断重叠是否能够缩短关键路径。本课题据此研究融合分区、跨 Kernel 依赖重叠及其运行时选择，并通过不同 batch、序列长度和硬件资源条件下的实验分析其适用范围。

三 课题主要研究内容与预期目标

3.1 总体目标

本课题拟设计并实现一个面向低延迟 LLM 推理的边界协同优化原型。系统以 PyTorch/编译器图、kernel 元信息和实际 GPU trace 为输入，识别关键路径上的高代价边界；生成不同粒度的融合分区，并为被保留的边界分析跨 kernel 依赖、构造 DACOS overlap；最后在动态 shape、资源竞争或收益不确定时选择融合范围与跨边界执行方案，或回退到成熟 library、TorchInductor 和 CUDA Graph 路径。

研究不以构建通用深度学习编译器为目标，而聚焦 decoder-only LLM 中频繁出现且对低延迟敏感的算子组合。预期覆盖 residual/RMSNorm、RoPE、GEMM epilogue/prologue、activation、attention wrapper、KV metadata 和 sampling 前处理等典型区域；其中具体纳入融合实现的模式将由基线 profiling、合法性和收益分析确定，不预设所有模式都应融合。

3.2 拟解决的关键问题

1. 如何建立连接编译图与设备执行 trace 的边界表示，使系统同时知道相邻 kernel 的算子语义、依赖张量、shape/layout、资源使用 and 实际时间贡献？

2. 如何判断一个边界“可以融合”且“应当融合”？除语义合法性外，如何量化中间访存和 launch 的节省，以及寄存器、shared memory、occupancy、代码尺寸和编译特化带来的代价？当保留边界并采用 DACOS 更优时，如何做出反向拆分选择？

3. 对无法融合的生产者-消费者边界，如何证明后继 kernel 的一段计算不依赖前驱输出，并利用 PDL 在不改变内存可见性和数值语义的前提下提前执行？

4. 当 workload 的 batch、sequence length、KV length、mask 或 backend 路径变化时，如何在融合、图重放、多流、DACOS 和原始路径之间稳定选择，并避免 guard miss、资源竞争和负收益？

5. 如何设计共同强基线和正交消融，分别解释本文融合方法、DACOS 方法及二者组合后的端到端收益，并避免把已有 library/compiler 优化计入任一项贡献？

3.3 主要研究内容

第一，研究 LLM kernel graph 的边界表征与瓶颈识别。本文将把框架图中的 operator/subgraph 与 Nsight Systems、Nsight Compute 或 CUPTI 采集的 kernel timeline 对齐，为每条边界记录生产者、消费者、读写集合、中间张量字节数、shape/layout、stream、launch gap、依赖等待和硬件资源。边界表示既用于筛选关键路径，也用于区分 host launch、全局内存物化、真实数据依赖和资源饱和等不同成因。

第二，研究面向 LLM 常见算子模式的融合分区与反向选择。系统从边界表示中识别 iteration space 兼容的 elementwise/reduction 邻域、GEMM epilogue/prologue 以及可由现有高性能库替换的子图，为同一局部图生成不同粒度的 Triton、CUTLASS 或自定义 CUDA 融合候选。决策既估计减少的 launch 与中间显存访问，也约束寄存器、shared memory、occupancy、编译版本数和动态 shape guard；同时保留“局部融合后拆分”与“完全不融合”的候选。研究重点不是融合模板的数量，而是能否解释为什么某个边界应被消除或有意保留。

第三，研究依赖感知跨 kernel 边界优化。本文将以 DACOS 为基础，对执行图中的生产者-消费者边进行依赖分析，将后继 kernel 分解为可提前执行阶段和依赖主体阶段，联合选择前驱触发位置、预取数据量和存储层级。该部分既在共同强基线上独立评价 DACOS，也研究它与本文融合模块的接口：融合改变图结构后重新分析跨 kernel 边；同一边界同时具有融合与 overlap 候选时分别评测；独立前缀过短或资源竞争严重时自动禁用。

第四，研究融合分区与跨边界重叠的联合决策。系统不预设“融合优先、重叠补充”的固定顺序，而是把融合图分区、边界消除收益、跨边界隐藏收益、资源变化和工程约束放入统一候选比较。选择器既可以合并两个 region，也可以反向拆开一个技术上可融合的 region，并对新出现的边界应用 DACOS。这里的“反向拆分”发生在图分区和代码生成之前，是保留候选边界，而不是对已经编译完成的 library kernel 做逆向分解。对于执行图中的不同区域，可以同时采用两种方法：对于稳定 kernel 序列，结合 CUDA Graph 降低 host 侧成本；对于真正独立的分支，使用 multi-stream。最终形成按 shape 和运行状态选择的执行计划缓存。

第五，完成系统原型与端到端验证。原型将与 PyTorch/TorchInductor 或一个可控的 LLM 执行框架连接，在 layer-level 和 model-level 两个尺度进行实验。Layer-level 实验解释边界合法性、内存流量和资源变化；model-level 实验评价 TTFT、TPOT 和端到端延迟，并观察局部收益能否沿关键路径累积。

3.4 预期创新点

1. 提出连接算子语义、编译信息与 GPU timeline 的 LLM 边界表示，使融合机会和跨 kernel 依赖重叠机会能够在同一对象上分析，而不是分别由编译器和运行时孤立处理。

2. 提出面向低延迟 LLM 常见模式的融合分区方法，生成不同粒度的 fused-kernel 候选，并综合中间数据物化、launch 节省、硬件资源和动态特化成本决定边界的消除或保留。

3. 在 DACOS 的依赖感知跨 kernel overlap 基础上，完善边界准入、资源保护和动态回退机制，使其作为独立优化路线与 fused-library、TorchInductor 和 CUDA Graph 公平组合。

4. 构建融合分区与跨 kernel 重叠协同的执行选择方法，支持对可融合区域

的反向拆分,并通过边界级消融和端到端实验说明更大融合、保留边界和 DACOS 重叠之间的取舍。

四 拟采用的研究方法、技术路线、实验方案及可行性分析

4.1 总体研究方法

本文采用“测量驱动的问题识别、语义约束下的代码变换、硬件反馈下的方案选择”三阶段方法。首先在真实模型执行中定位关键边界,避免仅凭算子名称假设瓶颈;其次依据数据依赖、控制依赖和内存可见性证明变换合法;最后通过资源模型和硬件测量判断变换是否有收益。该方法把 correctness、performance 和 deployability 分开处理:语义正确是候选成立的前提,性能收益由无 profiler 的成对测量确认,动态未命中和负收益场景必须有可复现的回退路径。

4.2 边界表示与分析方法

对执行图中的相邻 kernel K_i 和 K_j , 本文将边界表示为

$$B_{i,j} = \langle D_{i,j}, S_i, S_j, M_{i,j}, R_i, R_j, T_{i,j} \rangle,$$

其中 $D_{i,j}$ 表示数据和控制依赖, S_i, S_j 表示 shape、layout、dtype 与算子语义, $M_{i,j}$ 表示中间数据的物化和访问, R_i, R_j 表示资源使用, $T_{i,j}$ 表示实际时间线特征。编译图提供语义与依赖, kernel 元信息提供资源与代码结构, trace 提供运行时间和关键路径位置。三类信息通过 correlation id、NVTX region 或显式标注对齐。

在此表示上,系统先排除不在关键路径或成本可忽略的边界,再进行可融合性分析。可融合性不是单一布尔值,而包含语义合法性、调度兼容性和收益可行性三层。语义合法性检查随机数顺序、精度、reduction 顺序和 alias; 调度兼容性检查 iteration space、同步范围和 layout; 收益可行性检查减少的中间流量与 launch 是否足以补偿资源增长。

4.3 选择性算子融合方法

对一个融合候选区域 G_f , 本文使用如下形式分析预期收益:

$$\Delta T_{\text{fusion}} = T_{\text{launch}} + T_{\text{materialize}} - T_{\text{extra_compute}} - T_{\text{resource}} - T_{\text{specialize}}.$$

T_{launch} 表示被消除的启动成本, $T_{\text{materialize}}$ 表示被消除的中间数据写回与重读成本。 $T_{\text{extra_compute}}$ 表示变换引入的重复计算或同步, T_{resource} 表示寄存器、shared memory、occupancy 和 cache 压力造成的性能损失, $T_{\text{specialize}}$ 表示编译、autotune 和版本缓存成本在请求上的摊销。该模型不只用于接受融合候选; 当资源和特化代价超过被消除的边界成本时, 它应主动保留边界, 并把该方案送入跨 kernel 优化阶段。

进一步地, 将执行图的融合分区记为 P , 保留边界上的执行策略记为 S , 联合选择可以写为

$$(P^*, S^*) = \arg \min_{P, S} (T_{\text{critical}}(P, S) + \lambda C_{\text{compile}}(P) + \mu C_{\text{guard}}(P) + \nu C_{\text{resource}}(P, S)).$$

这里 T_{critical} 是实际关键路径延迟, 其他项分别约束编译与缓存成本、动态 guard 风险和资源干扰。该目标使“融合更多”和“保留边界后重叠”成为可以比较的候选, 而不是预先指定的先后步骤。成本模型只负责缩小搜索空间, 最终方案仍由离线硬件测量校准。

实现上优先选择三类模式。第一类是具有共同 iteration space 的 elementwise chain, 例如 bias、activation、residual、cast 和部分 mask 处理; 第二类是 GEMM epilogue/prologue 与 reduction 邻域, 例如 residual-add/RMSNorm、RMSNorm/Linear 和 MLP activation; 第三类是库替换或局部 wrapper 融合, 例如 RoPE、attention 前后处理和 sampling 前处理。不同模式分别采用 Triton、CUTLASS epilogue 或 CUDA 模板。对于 mask 和动态 shape, 系统根据 mask 的作用方式区分静态广播、shape-dynamic 和 value-dependent 情形: 前两类尽量通过符号 shape 或 bucket 覆盖, 涉及数据依赖控制流和动态输出的路径保留边界或回退。

4.4 依赖感知跨 Kernel 边界的 DACOS 方法

对执行计划中保留的依赖边 (K_i, K_j) , DACOS 将后继分解为

$$K_j = K_j^{\text{ind}} \cup K_j^{\text{dep}},$$

其中 K_j^{ind} 的读集合不包含 K_i 的输出, 可以在依赖数据可见前执行; K_j^{dep} 包含首次及后续依赖使用, 必须位于同步点之后。合法性分析要求同步点支配所有依赖读取, 同时保证提前执行的 load、descriptor 和地址计算在生命周期、alias 和异常行为上与原程序一致。

DACOS 的配置包含前驱触发阶段、独立工作量和存储层级。较早触发能够扩大 overlap window，但会增加与前驱尾部的共驻竞争；较多预取可能缩短后继主体，却可能污染 L2 或占用 shared memory。因此，成本模型联合估计可隐藏时间、预取驻留收益、未隐藏开销和资源干扰，并按边界和 shape bucket 选择配置。若后继独立前缀过短、前驱资源饱和或 PDL 调度没有形成有效重叠，系统回退到共同强基线；这项判断不依赖本文融合模块是否启用。

4.5 协同决策与运行时

系统首先建立共同基线，确认成熟 library 或编译器已经提供的优化；随后从高代价区域生成多种图分区，包括默认分区、更大融合分区和主动保留边界的较小融合分区。对每个分区，融合候选检查区域语义、数据布局和资源可行性，DACOS 候选检查保留边界的跨 kernel 依赖与后继独立前缀；无数据依赖的分支另行生成 multi-stream 候选。选择器既可以在同一边界上比较融合与保留，也可以在不同区域组合融合与 DACOS。CUDA Graph 位于 kernel 组织之后，用于重放稳定的最终执行计划，而不是被视为两项方法的替代品。

运行时按照模型、dtype、batch、sequence/KV bucket、layout、mask 类型和 backend 选择已校准计划。计划缓存只保存经过 correctness 与性能验证的版本；guard miss、动态控制流、显存压力变化或 backend 切换时回退到原 library/TorchInductor 路径。该设计的目标不是让每条边界都被优化，而是保证系统只在能够解释并验证收益的区域改变执行。

4.6 实验方案

实验将包含算子组合、decoder layer 和完整模型三个尺度。算子组合实验用于隔离残差与归一化、归一化与线性层、RoPE 与 attention、GEMM 与激活、attention 与输出投影、KV metadata 以及 logits 与 sampling 等边界；decoder layer 实验用于观察多条边界连续出现时的关键路径变化；完整模型实验选择 Qwen、LLaMA 或其他可复现 decoder-only 模型，分别覆盖 prefill、decode 和短生成。

Workload 将系统扫描 batch size、prompt length、decode length 和 KV length，并纳入常见 dtype、attention backend、静态/动态 shape、不同 mask 类型和 cache layout。主要关注 batch 1-8、短增量和 decode，但同时加入较大 batch、长序列与

资源饱和场景，用于确定方法失效点。Agent/RAG trace 只用于提供请求分布和状态变化背景，核心机制结论来自可控的 GPU 执行实验。

基线按贡献归因设置。PyTorch eager 只用于诊断原始碎片；成熟 fused library、TorchInductor 与 CUDA Graph 构成共同强基线。在同一模型、shape 和 backend 配置上，分别测试“共同基线加本文融合决策”“共同基线加 DACOS”和“共同基线加融合决策与 DACOS”，从而独立测量两项工作的收益及组合效果。融合部分还需比较 compiler default、greedy/maximal fusion、只考虑中间访存的局部 cost model，以及本文考虑资源与跨边界机会的分区方法。对于同一边界同时存在融合和 DACOS 候选的情况，增加二者直接比较；对于可实现的典型区域，参考 megakernel 或手工融合版本作为局部上界。

评价指标分为四组。第一组是用户可见性能，包括 TTFT、TPOT、端到端延迟和 P50/P90/P99；第二组是边界结构，包括 kernel 数量、关键路径边界时间、消除的中间字节数和残余边界比例；第三组是硬件行为，包括 SM 利用率、achieved occupancy、DRAM/L2 流量、寄存器与 shared memory 使用；第四组是工程稳定性，包括编译时间、版本数、guard 命中率、回退比例和数值正确性。

消融实验将分别回答以下问题：compiler default 与 maximal fusion 是否总能得到最低延迟；去掉融合资源项后是否出现 occupancy 下降；去掉动态特化项后是否出现版本膨胀；哪些区域由本文选择器主动保留边界；这些边界采用 DACOS 后是否优于 maximal fusion；在相同共同基线上，PDL-only 与完整 DACOS 的差异；去掉资源 guard 后是否产生负收益；动态 shape、mask、KV length 和 batch 变化如何改变图分区；在 fully fused、长序列和资源饱和区域，系统是否能够正确保持 no-op 或回退。

所有性能结论使用关闭 profiler 的成对重复测量，并报告分布而非单次最优值；profiler 只用于解释边界位置、资源变化和 overlap 是否发生。正确性检查覆盖输出张量误差、生成 token、KV cache 更新、随机采样状态和不同 shape 路径。实验固定软件版本、GPU 时钟策略、warmup、随机种子和 backend 配置，并保留原始日志与生成脚本。

4.7 可行性分析

前期 DACOS 工作已经完成跨 Kernel 依赖分析、PDL 模板、成本模型和运行时设计，并在 H100、H20 及多个模型上进行了实验。上述实现可直接用于验证后继 Kernel 阶段划分与 PDL 执行机制；后续将在统一实验环境中复核性能，并补充资源竞争、动态回退以及融合改变执行图后的重新分析。

融合部分拟从若干高频 LLM 局部图入手，使用 Triton、CUTLASS 和 TorchInductor 完成不同融合粒度的候选实现。通过限制算子模式和输入形状范围，可以控制代码生成与调优空间；通过比较默认执行、局部融合和较大融合区域，可以逐步校准数据移动与资源占用模型，并据此扩展到更多算子组合。

本人具备 CUDA、Triton、PyTorch、Nsight Systems/Compute 和 LLM serving profiling 经验，已参与 DACOS 工作，并在实习中接触 attention、block-sparse kernel、SM90/SM100 和分布式并行训练相关工程优化。这些经验能够支持 kernel 代码实现、硬件资源分析和系统实验。涉及实习项目的内容将只使用可公开、可复现的方法和数据，不把公司内部实现或未授权结果作为学位论文证据。

本课题的主要风险有三类。第一，成熟编译器和 library 已经覆盖大量简单融合，新增融合空间可能比预期小；对此需要从关键路径中选择仍有结构性优化空间的模式，并把融合代码生成和资源决策做深，而不是追求模式数量。第二，DACOS 与融合可能在部分边界上覆盖同一收益来源；实验将通过共同基线、直接候选比较和正交消融区分各自贡献，同时报告适用区间。第三，统一系统可能因组合空间过大而难以完成；实现上采用离线候选生成、少量 shape bucket 和保守运行时缓存，优先保证核心路径可复现。

五 已有科研基础与所需科研条件

5.1 已有科研基础

本人已完成 DACOS 论文的主要研究工作。现有稿件将短序列 LLM 执行中的依赖边界作为优化对象，提出依赖分析、operator-aware overlap stage、层次化预取和成本模型选择，并形成 H100/H20 上的模型与子图实验。该工作已经回答了“在不消除 kernel 边界时，如何利用后继独立工作隐藏等待”这一问题，是本课题第二部分的直接基础。

前期已完成 trace 解析、边界标注、实验配置和统计检查等基础工具，可用于连接编译图与 GPU timeline。已有的 RMSNorm、GEMM、Attention 和 sampling 边界分析为筛选融合模式提供了起点；下一阶段将补充真实模型执行数据、融合候选实现和成对性能实验。

5.2 所需科研条件

课题需要支持 PDL 的 NVIDIA GPU，主要实验平台为 Hopper/SM90，并在条件允许时补充后续架构；软件环境包括 CUDA、PyTorch、Triton、CUTLASS、TorchInductor、Nsight Systems 和 Nsight Compute。端到端实验需要可修改的 LLM 推理代码和稳定的模型权重、数据集及 workload replay。实验过程需记录 driver、CUDA、编译器、kernel library、模型配置、dtype、时钟和功耗策略。

设备资源不足时，将采用“microbenchmark 证明机制、decoder layer 解释组合、端到端确认价值”的分层策略。融合与 DACOS 的核心判断均可以在 layer-level 完成，model-level 主要验证收益能否沿关键路径累积。该策略能够避免把所有研究风险集中在完整 serving 系统集成上。

六 研究工作计划与进度安排

- 2026 年 7–8 月：完成开题报告与论文问题定义，复核 DACOS Euro-Par 实验和强融合基线；建立编译图、kernel 元信息与 GPU trace 的边界对齐流程。
- 2026 年 9–10 月：完成典型 LLM 边界 profiling，确定首批融合模式；实现局部融合候选、正确性检查和基础成本估计。
- 2026 年 11–12 月：完成融合模块的 shape/resource 筛选与回退；将融合与 DACOS 接入共同执行框架，完成共同基线、融合-only、DACOS-only 和协同配置的对照实验。
- 2027 年 1 月：完成动态 shape、mask、KV length、batch 与资源竞争消融，补充 fully fused、长序列和负收益案例。
- 2027 年 2 月：完成 model-level 实验、统计复核和论文主要图表，形成学位论文初稿。
- 2027 年 3 月以后：根据导师意见修改论文，完成参考文献、复现材料、盲审和答辩准备。

参考文献

- Agrawal A, Kedia N, Panwar A, et al. Taming throughput-latency tradeoff in LLM inference with sarathi-serve [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation. 2024: 117-134. <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- Bian Z, et al. Tokencake: A KV-cache-centric serving framework for LLM-based multi-agent applications [EB/OL]. 2025. <https://arxiv.org/abs/2510.18586>.
- Chen T, Moreau T, Jiang Z, et al. TVM: An automated end-to-end optimizing compiler for deep learning [C/OL]//13th USENIX Symposium on Operating Systems Design and Implementation. 2018: 578-594. <https://www.usenix.org/conference/osdi18/presentation/chen>.
- Cheng X, Zhang Z, Zhou Y, et al. MPK: A compiler and runtime for mega-kernelizing tensor programs [EB/OL]. 2025. <https://arxiv.org/abs/2512.22219>.
- Dao T. Flashattention-2: Faster attention with better parallelism and work partitioning [EB/OL]. 2023. <https://arxiv.org/abs/2307.08691>.
- Dao T, Fu D Y, Ermon S, et al. Flashattention: Fast and memory-efficient exact attention with IO-awareness [EB/OL]. 2022. <https://arxiv.org/abs/2205.14135>.
- Dong W, et al. Ada-mk: Adaptive megakernel optimization via automated DAG-based search for LLM inference [EB/OL]. 2026. <https://arxiv.org/abs/2605.11581>.
- Hao Z, Li G, Luo F, et al. DACOS: Dependency-aware cross-kernel overlapping for optimizing short-sequence workloads in LLM applications [Z]. 2026.
- He W, Jiang Y, Zhao P, et al. Efficient multi-round LLM inference over disaggregated serving [EB/OL]. 2026. <https://arxiv.org/abs/2602.14516>.
- Jin H, Hou B, Wang G, et al. Event tensor: A unified abstraction for compiling dynamic megakernel [EB/OL]. 2026. <https://arxiv.org/abs/2604.13327>.
- Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model serving with PagedAttention [C/OL]//Proceedings of the 29th Symposium on Operating Systems Principles. 2023: 611-626. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- Li B, Jiang Y, Gadepally V, et al. LLM inference serving: Survey of recent advances and opportunities [EB/OL]. 2024. <https://arxiv.org/abs/2407.12391>.
- Li J, Xu J, Huang S, et al. Large language model inference acceleration: A comprehensive hardware perspective [EB/OL]. 2024. <https://arxiv.org/abs/2410.04466>.
- Luo F, et al. Agentix: An efficient serving engine for LLM agents as general programs [EB/OL]. 2026. <https://www.usenix.org/conference/nsdi26/presentation/luo>.
- Luo X, Liu Z, Zhou Y, et al. Clusterfusion: Expanding operator fusion scope for LLM inference via cluster-level collective primitive [EB/OL]. 2025. <https://arxiv.org/abs/2508.18850>.

- Niu W, Guan J, Wang Y, et al. DNNFusion: Accelerating deep neural networks execution with advanced operator fusion [C/OL]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 883-898. DOI: [10.1145/3453483.3454083](https://doi.org/10.1145/3453483.3454083).
- Nrusimha A, Brandon W, Mishra M, et al. Flashformer: Whole-model kernels for efficient low-batch inference [EB/OL]. 2025. <https://arxiv.org/abs/2505.22758>.
- NVIDIA. CUDA programming guide: CUDA graphs [EB/OL]. 2026[2026-06-30]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs>.
- NVIDIA. CUTLASS 3.0 GEMM API [EB/OL]. 2026[2026-06-30]. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/gemm_api_3x.html.
- NVIDIA. CUDA programming guide: Programmatic dependent launch [EB/OL]. 2026 [2026-06-30]. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/programmatic-dependent-launch.html>.
- NVIDIA. NVIDIA TensorRT documentation: Layer fusion catalog [EB/OL]. 2026[2026-06-30]. <https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/fusion-catalog.html>.
- Pan J, Li G. A survey of LLM inference systems [EB/OL]. 2025. <https://arxiv.org/abs/2506.21901>.
- Patel P, Choukse E, Zhang C, et al. Splitwise: Efficient generative LLM inference using phase splitting [C/OL]//51st ACM/IEEE Annual International Symposium on Computer Architecture. IEEE, 2024: 118-132. <https://doi.org/10.1109/ISCA59077.2024.00019>.
- PyTorch Contributors. CUDA semantics: CUDA graphs [EB/OL]. 2026[2026-06-30]. <https://docs.pytorch.org/docs/stable/notes/cuda.html#cuda-graphs>.
- PyTorch Contributors. torch.compiler [EB/OL]. 2026[2026-06-30]. https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/torch.compiler.html.
- Qin Z, et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving [C/OL]//23rd USENIX Conference on File and Storage Technologies. 2025. <https://www.usenix.org/conference/fast25/presentation/qin>.
- Shah J, Bikshandi G, Zhang Y, et al. Flashattention-3: Fast and accurate attention with asynchrony and low-precision [EB/OL]. 2024. <https://arxiv.org/abs/2407.08608>.
- Shi Y, Yang Z, Xue J, et al. Welder: Scheduling deep learning memory access via tile-graph [C/OL]//17th USENIX Symposium on Operating Systems Design and Implementation. 2023: 701-718. <https://www.usenix.org/conference/osdi23/presentation/shi>.
- Sui Y, Zhao H, Ma R, et al. Parallelizing tool execution and LLM generation for low-latency agent serving [EB/OL]. 2026. <https://arxiv.org/abs/2603.18897>.
- Tillet P, Kung H T, Cox D D. Triton: An intermediate language and compiler for tiled neural network computations [C/OL]//Proceedings of the 3rd ACM SIGPLAN International Workshop on

- Machine Learning and Programming Languages. 2019: 10-19. <https://doi.org/10.1145/3315508.3329973>.
- Vellaisamy P, Labonte T, Chakraborty S, et al. Characterizing and optimizing LLM inference workloads on CPU-GPU coupled architectures [EB/OL]. 2025. <https://arxiv.org/abs/2504.11750>.
- Vellaisamy P, Tripathi S, Natarajan V, et al. Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition [EB/OL]. 2026. <https://arxiv.org/abs/2603.12465>.
- vLLM Contributors. vllm design document: CUDA graphs [EB/OL]. 2026[2026-06-30]. https://docs.vllm.ai/en/v0.14.0/design/cuda_graphs/.
- Ye Z, et al. Flashinfer: Efficient and customizable attention engine for LLM inference serving [EB/OL]. 2025. <https://arxiv.org/abs/2501.01005>.
- Yu G I, Jeong J S, Kim G W, et al. Orca: A distributed serving system for transformer-based generative models [C/OL]//16th USENIX Symposium on Operating Systems Design and Implementation. 2022: 521-538. <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Zhong Y, Liu S, Chen J, et al. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation. 2024: 193-210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.